A Phase Vocoder Approach To Pitch Shifting

Sam Smith

April 1, 2026

1 Introduction

The manipulation of pitch in digital signals is not achieved by simply scaling frequencies, but through structured transformations in the time-frequency domain that reshape signals in a variety of ways.

1.1 Pitch Shifting

Pitch shifting is a desirable signal processing effect for a variety of practical reasons. Historically, pitch manipulation emerged from research in vocal processing (giving rise to the term 'vocoder' a concatenation of the words 'vocal' and 'coder') [2]. Today, pitch-shifting algorithms are widely used in music production, film, voice acting, and media. Specifically in music production, digital vocal correction tools such as autotuning rely on pitch modification techniques to adjust a singer's intonation while preserving timing and vocal characteristics.

Pitch shifting is not limited to vocal signals, however, it is also commonly applied to instrumental audio. For example, electric guitarists use pitch-shifting pedals and digital audio plugins to alter tuning without physically adjusting string tension. This capability allows musicians to explore alternate tunings without modifying their instrument hardware or spending vast amounts of time retuning extended ranged guitars. Such applications highlight the broad practical demand for high-quality, real-time pitch-shifting algorithms.

1.2 The Phase Vocoder Approach

The phase vocoder is a signal processing technique used for both time-scale and pitch modifications. This algorithm was chosen to be the core processing algorithm for this project due to the fact that the family of vocoder algorithms is known for their reliability, ability to handle polyphonic signals, and their widespread adoption many modern signal processing applications. Furthermore, the phase vocoder is a real-time algorithm, meaning it has the capability to execute, in its entirety, in an unperceivable amount of time. Hence, this algorithm, along with the JUCE audio plugin framework, will provide a basis for creating a real-time pitch-shifting audio plugin.

1.3 Outline of Remaining Chapters

This paper begins with a comprehensive overview of the mathematics needed to understand the phase vocoder algorithm. It will begin with an explanation of the Discrete Fourier Transform, the backbone of signal processing, then we will examine windowing, why we need it and what problems it solves. Finally, we will examine the Short-Time Fourier Transform (STFT) which is the core analysis algorithm used in the phase vocoder.

The next section will provide a deep intuition for the phase vocoder algorithm where the goal is to connect the mathematical concepts highlighted in the background section to the processing chain that comprises the phase vocoder algorithm. The section will be divided based on the three fundamental steps that make up the phase vocoder algorithm: analysis, modification and synthesis.

Next, we will translate the mathematical concepts built up by the previous sections into a coherent program architecture. The audio plugin will be constructed in the JUCE framework, an open-source c++ framework designed to build audio plugins and and applications,

Finally, the following will evaluate output quality using metrics such as tonality and spectral flatness (noise) will be evaluated against to test how good the phase vocoder is at modifying a given input signal.

The conclusion follows the results chapter and will primarly focus on results, future work and optimizations that can be done on this project. Additionally, this section will provide insight into any errors or unoptimized processes in this project and offer any additional improvements or next steps.

2 Background

2.1 Discrete Fourier Transform

The Discrete Fourier Transform (DFT) is a fundamental tool used in digital signal processing that enables sophisticated analysis and synthesis of discrete-time signals [6]. The DFT transforms a signal from the time domain into its frequency-domain representation, allowing the signal to be expressed as a sum of sinusoids [6]. The DFT accepts a finite sequence of N discrete samples and produces N complex-valued outputs, each representing the signal's magnitude and phase at one of N evenly-spaced frequency bins. The output of the DFT can be better understood as a sequence of N complex numbers representing amplitude and phase for each analysis frequency [7]. Mathematically, the DFT is defined as:

$$X[m] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi n \frac{m}{N}} \tag{1}$$

where N is the number of samples from the input signal $x[n]$, n denotes the time-domain index, m represents the frequency bin index, n is the index of the input sequenc in the time-domain and $j = \sqrt{-1}$. Each coeffecient in $X[m]$ is complex-valued and can be expressed in terms of magnitude and phase, which describe the contribution of a corresponding frequency component to the overall signal [7].

- Talk about inverse DFT

2.2 Windowing

Windowing is a function used to isolate parts of a discrete time signal so that analysis on the signal can be performed on contiguous blocks [1]. More specifically, the input signal is multiplied by a windowing function whose domain is nonzero for a particular range. This results in a small localized snapshot of the input signal, and every other value outside of the windows domain is reduced to zero [10]. Figure 1 below demonstrates the functionality of a window function.

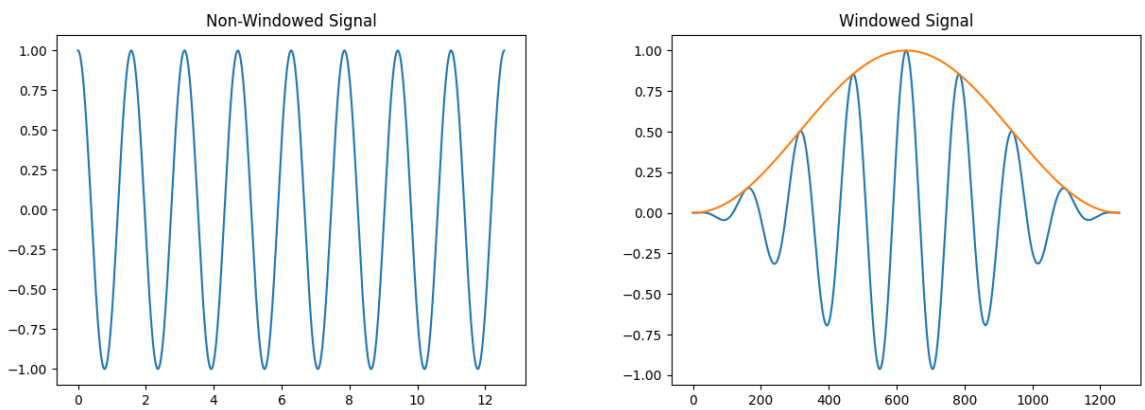

Figure 1: After applying a window function (highlighted by the orange cosine curve), the signal is tapered at the edges, resulting in a temporally localized signal.

As depicted by Figure 1, after applying the window function to the example signal, the edges of the signal tend toward zero. In this example, the Hann window function was used to shape the signal, however, there are multiple windowing functions such as the Rectangle window function, Triangle window function and Hamming window function [7]. For this project, the Hanning window will be employed as it yields a more smooth, continuous signal during the resynthesis step [3].

- Further explain how windowing prevents spectral leakage and why we need it (prevents artifacts, fft assumes signal is periodic, does not work if not continuous at edges)

2.3 Short-Time Fourier Transform

The Short-Time Fourier Transform (STFT) is a time-frequency representation used to compute the DFT of contiguous windowed segments of a signal. This is accomplished by multiplying the signal by a windowing function so that only a small portion of the signal is nonzero [2]. Mathematically, it is similar to equation (1) but with a few modifications: the addition of a windowing function, the domain of the input signal, and how the output is interpreted.

Below is the STFT:

$$X[m,k] = \sum_{n=0}^{N-1} x[n+kH]w[n]e^{-2\pi jmn/N} \tag{2}$$

Where $X[m,k]$ is the 2-dimensional output array indexed by both frequency bin and time frame and $x[n+kH]$ is the section of the input signal determined by frame index and hop size [10]. The outcome of the STFT is a representation that transforms a signal from the time domain to the frequency domain. In this form, the signal's frequency components can be systematically modified and recombined to reconstruct a new signal.

- talk about inverse STFT

3 Related Works

3.1 Classical Phase Vocoder Approaches

Pitch-shifting has been approached by a number of time–frequency methods. Early methods, such as Dolson's (1986), employed the use of the Short-Time Fourier Transform in conjunction with the phase vocoder algorithm. In particular, Dolson describes a pitch-shifting method that relies on time-scale manipulation to modify the pitch of a given signal [2]. For example, by stretching a signal, making it longer in the time domain, and then rescaling it back to its original length, the overall frequency content of the signal is effectively decreased, which corresponds to lowering the signal's pitch. Dolson acknowledges, however, that this approach is imperfect: specific constraints must be enforced when performing a pitch shift, as failing to do so may introduce unwanted artifacts into the signal or result in a reconstructed signal that has lost information [2]. While effective for monophonic (single note) pitch scaling, this classical time-scale modification framework treats spectral bins independently, which can lead to phase incoherence and unwanted artifacts. These limitations motivated subsequent refinements to the phase vocoder algorithm.

3.2 Phase Vocoder Optimizations

Unlike the original process chain described by Dolson (1986), which applies a uniform scaling factor to all spectral components, later work by Laroche and Dolson (1999) introduces refinements that improve frequency coherence [4]. In particular, they highlight a new peak-detection paradigm, where peaks refer to how much a particular frequency is in the input signal. Once the peaks are detected, they can be shifted around by a pitch-factor and unlike the version described by Dolson (1986), the pitch-factor is not restricted to an integer amount [4].

In addition to optimizing frequency resolution, modern implementations of the phase vocoder shift away from the standard two-step timescale and resample approach and adopt an analysis and resynthesis paradigm [3]. The new approach leverages phase information obtained from the STFT step to more accurately reconstruct the modified signal [3]. More specifically, a phase adjustment value can be derived from the frequency bins across two

frames and when added to the original frequency approximation the accuracy of the frequency estimation is increased [5].

3.3 Alternative Pitch-Shifting Methods

The prior sections highlight the phase vocoder approach to pitch-shifting, a frequency-domain algorithm that converts a signal from the time domain to the frequency domain to compute the pitch shift. An alternative approach uses time-scale modification (TSM) algorithms, which operate entirely in the time domain [8]. A common algorithm belonging to the TSM family is the Overlap-Add (OLA) algorithm [10].

The idea behind these algorithms is that pitch shifting and time scaling are essentially the same; meaning if you have one, either a pitch shifter or time scaler, then you effectively have the other. OLA makes use of this paradigm. The algorithm first partitions the input signal into frames, much like the STFT. Then, if the goal is to pitch down the signal, when resynthesizing the signal it is constructed in such a way that extra space is added between the reconstructed frames, increasing the length of the signal. On the contrary, if the goal is to make the signal's pitch higher, then the algorithm reconstructs the signal in such a way where the space between contiguous frames is reduced, effectively shortening the signal's length [10].

Since TSM algorithms operate in the time domain and thus change the input signal's duration, either lengthening or shortening it, these algorithms must incorporate a resample step in order to convert the modified signal back to the original signal's duration [8]. Thus, these alternative algorithms are viable options; however, purely TSM-based algorithms are often outperformed by frequency-based algorithms [10].

4 Phase Vocoder

The phase vocoder is a name given to a group of algorithms designed to process an input signal in the frequency domain [9]. This paper specifically focuses on the application of pitch shifting, the process of either increasing or decreasing an audio signal's pitch. In particular, this work utilizes a phase vocoder-based pitch shifting algorithm, as it is a computationally efficient method for analyzing and modifying an input audio signal, and is the backbone of many real-time pitch shifting audio plugins used today [10].

The following subsections will explain the inner workings of the phase vocoder algorithm and its application to pitch shifting. The first subsection will explain the analysis step and how the STFT breaks an audio signal into frames. The second subsection will cover the modification step and explain how pitch shifting is computed in the frequency domain. Finally, the last subsection will explain how the modified signal is synthesized back into an output signal.

4.1 Analysis

The first step of the phase vocoder algorithm is the analysis step. The primary goal of this step is to break an input audio signal into a succession of overlapping frames so that

each frame is processed individually. In practice, the STFT is used to partition the audio signal into frames of length N. This is accomplished by multiplying the signal frame by the Hann window function. Once the signal frame has been isolated, the FFT is computed on the windowed signal chunk, converting it from the time domain to the frequency domain. To process the next frame, the analysis window moves ahead by H (hop size) number of samples.

The output of the analysis step is a collection of M frequency bins (see Section 2.3), where each bin contains a complex value representing the magnitude and phase information for a particular frequency [9]. At this point, the signal frame can be modified in the frequency domain, such as altering the pitch.

- Perform block-based processing to analyze just a chuck of the incoming audio buffer
- Apply the windowing function to reduce spectral leakage (explain in windowing section)
- The result will be a frequency-domain representation of the signal chunk, this is the correct representation for preforming frequency transformations

4.2 Modification

In this section I will explain how the phase algorithm uses the phase information obtained from the DFT to perform a more accurate frequency transformation. Additionally, the phase information is used to better reconstruct the signal in the resynthesis step

4.3 Synthesis

Once the signal frame has been modified, the IFFT converts the signal back to the time domain. Then the output is reconstructed using the Constant Overlap-Add (COLA) paradigm. A detailed explanation of COLA can be found here [11]. The key idea is that if samples in the window function, offset by H, add to a constant C, then the window possesses the "Constant Overlap-Add Property" [11]. The mathematical expression below demonstrates the COLA property:

$$\sum_{m=-\infty}^{\infty} w(n - mH) = 1, \quad \forall n \in \mathbb{Z} \tag{3}$$

This project employs the Hanning window function with a 75% overlap. To prove it possesses the COLA property we can graph several Hanning window functions with a 75% overlap and their corresponding overlap value. Figure 2 shows that by graphing five Hanning window functions with a 75% overlap satisfies the COLA property. Notice a 66.67% ($\frac{2}{3}$) overlap does not work as the sum of the samples fluctuates.

- FIGURE 2 HERE ONCE COMPLETE

To reconstruct the output signal, the values from the inverse STFT are first added into an output buffer. This buffer serves as temporary storage; it is not the final synthesized

audio signal. The output buffer is necessary because, due to overlapping windows ensured by the COLA property, certain samples are calculated as the sum of multiple windows. Once the samples have been correctly added in the buffer, a write pointer is used to read them sequentially, producing the final output signal.

- After performing the frequency transformation step, we take the inverse DFT of the signal to convert it back into the time-domain
- We also apply the windowing function again to ensure that when we recombine the signal the edges of contiguous blocks line up
- I will explain why then we need to multiply our discrete signal values by a windowing Correctin value

5 Methodology

In this section, I will be explaining how the phase vocoder algorithm is implemented in the JUCE framework. This section will be comprised of how JUCE operates on audio signals the data structures needed to implement the phase vocoder algorithm, such as circular buffers, how the DFT is computed in JUCE

Transition Sentences

Now that we have unveiled the innerworkings of the phase vocoder algorithm, we will see how it is implemented in the JUCE audio framework JUCE is an open-source cross platfrom C++ framework used to develop audio applications and plugins

- I built a pitch shifting audio plugin
- My goals were to build a real-time, polyphonic pitch shifting plugin that could pitch shift up to 5 semitones (downtune and uptune)
- I implemented this audio plugin in the JUCE framework, an open source framework used to develop audio applications and plugins
- The main plugin methods we are concerned with are PluginProcessor.PrepareToPlay(), PluginProcessor.releaseResources(), and pluginProcessor.processBlock()
- The main functionality of JUCE we are concerned with is the AudioBuffer. This is an array of discrete signal values that get's passed to the plugin class
- I will also explain circular buffers, as this is the main data structure that the phase vocoder employs. It is used in two places, the inputBuffer and outputBuffer

6 Results

- A successful pitch-shift audio plugin will produce a shifted audio signal with minimal artifacts introduced
- Additionally, the success will depend on how well it can handle multiple notes. Full chords, two power chords, fast-alternate picking etc.
- If time permits, I will test the audio clarity using one of the tests found here: [8]

7 Conclusion

References

[1] Barry, D., Dorran, D., and Coyle, E. Time and pitch scale modification: A real-time framework and tutorial.

[2] Dolson, M. The phase vocoder: A tutorial. *Computer Music Journal 10*, 4 (1986), 14–27.

[3] Götzen, A. D., Bernardini, N., and Arfib, D. Traditional (?) implementations of a phase-vocoder: The tricks of the trade. In *Proceedings of the COST G-6 Conference on Digital Audio Effects (DAFx-00)* (Verona, Italy, Dec. 2000), pp. 37–44.

[4] Laroche, J., and Dolson, M. New phase-vocoder techniques for pitch-shifting, harmonizing and other exotic effects. In *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)* (New Paltz, NY, USA, Oct. 1999), IEEE, pp. 91–94.

[5] Lim, K. A. An open-source phase vocoder with some novel visualizations. Capstone project, School of Informatics, Indiana University Bloomington, June 2007.

[6] Lyons, R. G. *Understanding Digital Signal Processing*, 3rd ed. Pearson, Upper Saddle River, NJ, USA, 2010.

[7] McFee, B. Digital signals theory, 2025.

[8] Rai, A., and Barkana, B. D. Analysis of three pitch-shifting algorithms for different musical instruments. In *2019 IEEE Long Island Systems, Applications and Technology Conference (LISAT)* (2019), IEEE, pp. 1–6.

[9] Reiss, J. D., and McPherson, A. *Audio Effects: Theory, Implementation and Application*. CRC Press, 2014. Accessed as PDF.

[10] Royer, T. Pitch-shifting algorithm design and applications in music. Master's thesis, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2019.

[11] Smith, J. O. *Spectral Audio Signal Processing*. 2011. Online book.